

# CLEAR: Capability-Aware Latent Ordering for Adaptive Interaction with Movable Objects

Anonymous Authors

**Abstract**—Planning physical interactions in cluttered environments requires reasoning about robot capabilities. For example, a legged robot may climb stairs, while a wheeled robot may need an alternative route. However, the ability to move objects does not ensure task completion, as the interaction order and resulting object configurations can block subsequent actions. To address this challenge, we propose a capability-aware planner that jointly determines *which* objects to interact with, *in which order*, and *how* to move them. Our planner, Capability-Aware Latent Ordering for Adaptive Interaction with Movable Objects (CLEAR), learns to select objects and propose alternative interaction orders from scene observations and robot structure. For each proposed sequence, it generates object references, conditioning each prediction on those planned for preceding interactions. The planner checks whether candidate plans are feasible in the proposed interaction order, selects the valid candidate with the fewest object interactions, and replans from updated observations during execution. Our approach is evaluated in three simulation environments: 2D grids, ordered manipulation, and maze navigation. The maze experiments assess the effect of robot-structure conditioning and the ability to replan when environmental perturbations block access to required objects. Code, videos, and results are available at: <https://anonymous.4open.science/w/clear-page-1209/>.

## I. INTRODUCTION

Robots operating in homes and workplaces perform tasks in spaces occupied by furniture, equipment, and other objects. A box can block a corridor, while objects on a table can occupy the space needed for manipulation. Moving selected objects can open a path for navigation or free space for a desired arrangement. However, moving one object can obstruct a subsequent interaction. Planning these changes requires considering how object motions affect the space available for completing the task.

We refer to planning object interactions for navigation and object arrangement as *Planning Among Movable Objects* (PAMO). Prior work has studied how object relocation enables navigation through clutter [1], [2]. Navigation tasks specify a destination, while arrangement tasks specify conditions on the resulting object configurations. Both tasks require selecting objects, determining the interaction order, and planning the motion of each selected object. The resulting plan must allow the robot to complete the required interactions and achieve the task goal. Because object relocation requires additional execution effort, we aim to achieve the goal with the minimum number of object interactions.

Even with the scene and goal fixed, robots with different capabilities can require different interaction plans. A robot capable of crossing a step can bypass a blocked corridor, whereas another embodiment must move objects to clear an

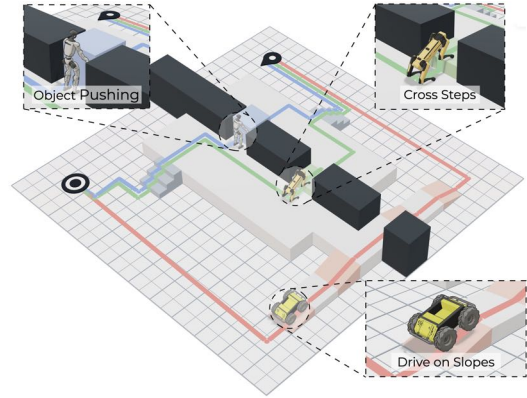


Fig. 1: **Different robot embodiments require different plans.** To reach the same goal, the humanoid pushes a movable object aside (blue), the quadruped crosses a step (green), and the wheeled mobile robot follows a sloped route (red). Our pipeline infers capabilities from robot structure and jointly predicts selected objects, interaction order, and target configurations.

alternative path. Reach and force limits further determine which objects can be moved and which motions can be executed. Feasibility also changes as preceding interactions alter the scene. For example, moving a box away from one target can obstruct the approach to another target or occupy space needed for a subsequent motion. Stilman and Kuffner [3] address these dependencies by constraining preceding object displacements using the requirements of subsequent motions. Object selection, interaction order, and continuous motion must therefore be considered together under the capabilities of the robot.

The same robot can achieve a task goal through different object selections and interaction orders. Different plans can achieve the same goal with the same number of object interactions, even when different objects are selected. Alternative plans provide options when execution errors or scene changes prevent the initially planned interactions. Generative models learn distributions over trajectories and action sequences to represent alternative behaviors [4], [5]. In PAMO, a learned distribution over interaction plans can represent alternative object selections, interaction orders, and the corresponding continuous motions.

To address these challenges, we present **Capability-Aware Latent OrdERing for Adaptive InteRAction with Movable Objects** (CLEAR). The main contribution is an embodiment-aware generative planner that learns object selection, inter-

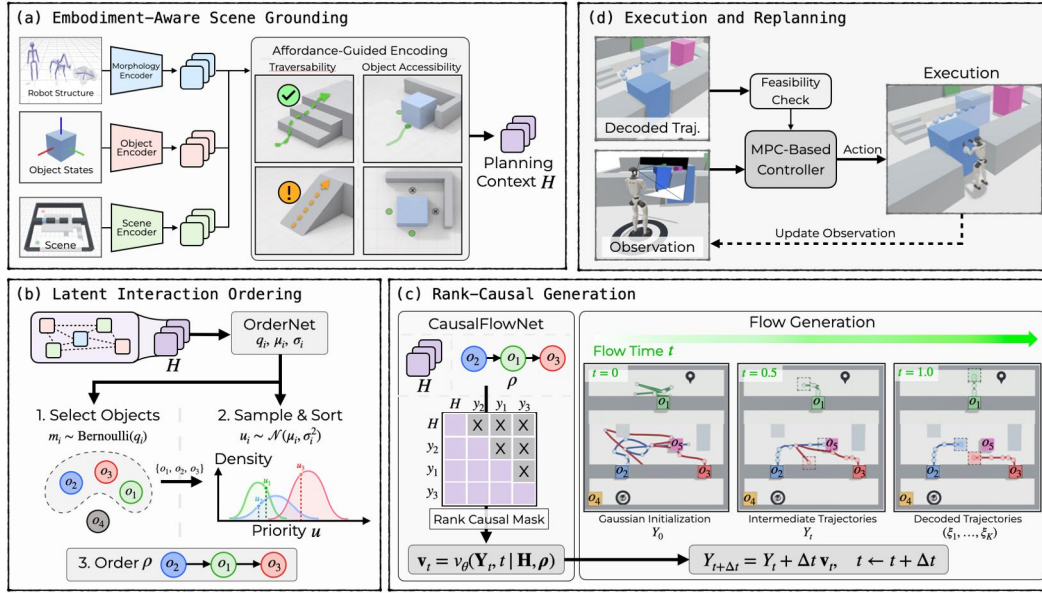


Fig. 2: **CLEAR overview.** Our approach combines robot capability prediction, object selection and ordering, and continuous interaction generation to plan physical interactions in cluttered environments. (a) **Affordance-Guided Encoding** combines robot, object, and scene features with capability predictions and object accessibility information to form the planning context  $\mathbf{H}$ . (b) **OrderNet** selects objects and determines candidate interaction orders  $\rho$  using learned selection probabilities and continuous latent scores. (c) **CausalFlowNet** generates continuous interaction parameters conditioned on  $\mathbf{H}$ , with rank-causal attention restricting dependencies to the same object and preceding interactions in  $\rho$ . (d) The planner checks candidate feasibility and selects a plan with the minimum number of object interactions among accepted candidates.

action order, and continuous interaction parameters within a common framework. Predicted interaction feasibility augments the scene representation with information about robot capabilities. Latent interaction ordering generates alternative object selections and orders. Conditional flow matching generates target configurations or trajectories for each sequence. An attention mask follows the interaction order and conditions each object prediction on preceding interactions. Among feasible candidates satisfying the task goal, the planner selects a plan with the minimum number of object interactions. Updated scene observations guide replanning of the remaining interactions during execution. We conduct four experiments across 2D grid, ordered manipulation, and maze navigation settings to evaluate interaction feasibility, plan diversity, embodiment conditioning, and recovery under environmental changes.

## II. RELATED WORK

### A. Planning Among Movable Objects

In planning among movable objects, relocating an object can alter the feasibility of subsequent interactions. For navigation, object relocation enables robot motion through clutter, as studied in Navigation Among Movable Obstacles (NAMO) [1]. Ren et al. [2] introduce PAMO\*, a search-based method for joint navigation and relocation planning without predefined final object configurations, while Interactive-FAR [6] updates navigation routes using interaction feedback. In object rearrangement, achieving the target

arrangement may require moving objects to temporary locations [7]. Stilman and Kuffner [3] constrain object relocations according to the requirements of subsequent interactions. ReloPush [8] plans rearrangements under nonholonomic motion and pushing constraints.

Beyond planning object motion, prior work considers the physical stability and semantic suitability of resulting object configurations. SPOTS [9] combines physical stability verification with semantic reasoning to identify suitable configurations. Our method instead generates alternative interaction plans conditioned on the scene and robot structure and selects a feasible plan with the fewest interactions.

### B. Generative Models for Robotic Planning

The success of generative models in language and vision has motivated research on generating diverse robot behaviors. Diffuser [4] and Diffusion Policy [5] learn distributions over trajectories and observation-conditioned action sequences, respectively. SMORE [10] generates diverse manipulation options for semi-autonomous teleoperation through quality-diversity sampling and latent-variable policies. Planned object motions must satisfy geometric constraints and allow subsequent interactions. StructDiffusion [11] generates object arrangements under language and geometric constraints, while Diffusion-CCSP [12] combines constraint-specific diffusion models to solve continuous constraints in task and motion planning. Generative Skill Chaining [13] composes learned skill distributions to generate long-horizon plans for a given skill sequence, while Generative Factor Chaining [14]

combines spatial constraints and skill transitions within a given symbolic plan graph. In contrast, our approach jointly learns object selection and interaction ordering to generate alternative plans. Conditional flow matching [15] generates continuous interaction parameters for each sequence, conditioned on the full observed scene and the generated parameters of preceding interactions.

### C. Embodiment-Aware Robot Planning

Robots with different structures can require different control strategies for the same task. NerveNet [16] and Shared Modular Policies [17] encode robot structure through message passing between body or actuator representations. MetaMorph [18], Body Transformer [19], and GET-Zero [20] incorporate morphology and body connectivity into Transformer-based control. Beyond controller transfer, XSkill [21] and shared goal-conditioned policies [22] study cross-embodiment learning through skill representations and experience from heterogeneous robots.

At the task level, robot capabilities determine which actions are feasible in the scene. CLARA [23] combines descriptions of robot functions and scene context to assess command feasibility and resolve ambiguity. E<sup>2</sup>-CARE [24] adapts reusable skill templates using constraints from robot embodiment and the environment. Our work uses capability information to determine the required object interactions for a fixed scene and goal. We condition scene representations on robot structure to predict traversal and interaction feasibility and support object selection and ordering.

## III. PROBLEM FORMULATION

### A. Embodiment-Aware Planning Among Movable Objects

We focus on the problem of planning object interactions to achieve navigation or arrangement goals. Let  $S$  denote a scene consisting of the observed environment geometry, the current robot state,  $N$  movable objects  $\{o_i\}_{i=1}^N$ , and a task goal  $G$ . Depending on the task,  $G$  specifies a navigation destination or a desired object arrangement. Each object  $o_i = (x_i, a_i)$  is described by configuration  $x_i$  and physical attributes  $a_i$ , including geometry and mass. The robot embodiment  $B$  is described by link, joint, and actuator properties. An interaction plan  $\pi = ((i_1, \xi_1), \dots, (i_K, \xi_K))$  is a sequence of  $K$  object interactions. The sequence  $\rho = (i_1, \dots, i_K)$  specifies the selected objects and the interaction order. For interaction  $k$ , we define  $\xi_k$  as the target configuration or trajectory of the selected object, depending on the task representation.

### B. Interaction Feasibility and Planning Objective

Interaction  $k$  is feasible if the robot can reach the selected object and perform the object motion required by  $\xi_k$  while satisfying the relevant geometric and physical constraints. Interaction feasibility is evaluated in the scene resulting from preceding object interactions. A plan  $\pi$  is valid if all required robot motions and object interactions are feasible and the final scene satisfies the task goal  $G$ . The planning objective is to find a valid plan with the minimum number

of object interactions. When the task goal  $G$  is achievable without object interaction, the minimum interaction count is  $K = 0$ . We learn a conditional distribution  $p_\theta(\pi | S, B)$  over interaction plans, where  $\theta$  denotes the trainable model parameters. The planner generates candidate plans from the learned distribution and selects a valid candidate with the minimum number of object interactions.

## IV. PROPOSED METHOD

Our approach learns the conditional plan distribution  $p_\theta(\pi | S, B)$  through embodiment-aware scene grounding, latent interaction ordering, and rank-causal generation (Fig. 2). *Affordance-Guided Encoding* forms a planning context  $\mathbf{H}$  from scene observations and robot structure (Sec. IV-A). *OrderNet* determines object selection and interaction order (Sec. IV-B), and *CausalFlowNet* generates continuous interaction parameters under the proposed order (Sec. IV-C). A low-level planner checks candidate feasibility and computes the required robot motions, with updated observations guiding control and replanning (Sec. IV-D).

### A. Embodiment-Aware Scene Grounding

To ground scene representations in robot capabilities, *Affordance-Guided Encoding* augments object features with predicted interaction availability. An embodiment encoder represents the structural properties of  $B$  as an embedding  $\mathbf{z}_B$  using shared feature encoding and average pooling. Conditioned on  $\mathbf{z}_B$  and local scene observations, the network predicts an affordance probability  $\hat{p}^{\text{aff}}$  for each candidate object interaction or terrain transition. A binary target  $b$  indicates whether the object interaction is available or the terrain transition is feasible. The network is trained by minimizing the binary cross-entropy loss  $\mathcal{L}_{\text{aff}} = \text{BCE}(\hat{p}^{\text{aff}}, b)$ .

Using the learned predictions, we augment object features with interaction probabilities and reachability information. When terrain geometry constrains robot movement, computing object reachability requires evaluating connectivity across the scene. Therefore, path search uses the predicted transition probabilities to determine which objects the robot can reach. The augmented object features are combined with the embodiment embedding  $\mathbf{z}_B$  and scene observations to construct the planning context  $\mathbf{H}$  for *OrderNet* and *CausalFlowNet*.

### B. Latent Interaction Ordering

*OrderNet* learns object selection and interaction order from the planning context  $\mathbf{H}$ . For each object, the network predicts a selection probability  $q_i$  and a Gaussian latent score distribution with mean  $\mu_i$  and standard deviation  $\sigma_i$ . Bernoulli sampling based on  $q_i$  determines the selected object set  $\mathcal{I}$ , and Gaussian sampling generates latent scores  $u_i$ . Sorting the selected scores from lowest to highest defines the interaction order [25],

$$\rho = \text{argsort}_{i \in \mathcal{I}} u_i, \quad (1)$$

where  $i$  denotes the object index and  $\mathcal{I}$  contains the selected object indices. Sampling object selections and latent scores generates multiple candidate interaction sequences for the

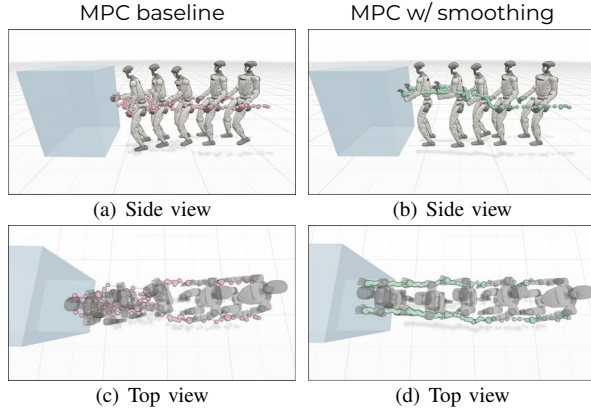


Fig. 3: **Comparison of low-level controllers for object interaction.** Side and top views compare the MPC baseline (a, c) with our controller (b, d). Our controller combines quality-diversity sampling, Cartesian tracking, and joint-command smoothing.

same scene and embodiment. Within each candidate, object selection considers the complete sequence, including objects the robot can reach after preceding interactions. Each candidate contains at most one interaction per object, and the same object can be selected again during replanning.

The network is trained to predict the object selections and interaction orders specified in the training data. The training objective is

$$\mathcal{L}_{\text{order}} = \mathcal{L}_{\text{sel}} + \mathcal{L}_{\text{rank}} + \lambda_{\text{KL}} \mathcal{L}_{\text{KL}}. \quad (2)$$

The selection loss  $\mathcal{L}_{\text{sel}}$  trains object selection using binary cross-entropy between the predicted probabilities  $q_i$  and the object selection labels. The weighted Bradley–Terry loss [26]  $\mathcal{L}_{\text{rank}}$  encourages lower scores for preceding objects, with greater weight on pairs close in the training order. Gaussian reparameterization enables gradient-based training of the score parameters. The regularizer  $\mathcal{L}_{\text{KL}}$  penalizes divergence of the predicted Gaussian distributions from a standard normal distribution.

### C. Rank-Causal Generation

Given the object set  $\mathcal{I}$  and interaction order  $\rho$  obtained in Sec. IV-B, CausalFlowNet generates continuous interaction parameters conditioned on  $\mathbf{H}$ . For each selected object, rank denotes its position in  $\rho$ . We represent object parameters by a matrix  $\mathbf{Y}_t \in \mathbb{R}^{N \times D}$ , where  $t \in [0, 1]$  denotes flow time. Each row  $\mathbf{y}_i(t)$  contains  $D$  coordinates for object  $o_i$ . The generator produces target configurations or complete object trajectories at  $t = 1$  (see Fig. 2(c)). The network predicts a velocity field  $\mathbf{v}_t = v_\theta(\mathbf{Y}_t, t | \mathbf{H}, \rho)$  to update these parameters. Rank-causal attention combines the token for each selected object with generated tokens from preceding ranks and the context  $\mathbf{H}$ . The context is shared because scene and robot information is available before generation. The attention mask blocks generated information from entering context tokens, preventing subsequent predictions from affecting preceding predictions indirectly through  $\mathbf{H}$ .

In the training phase, we use conditional flow matching [15] to train the velocity field. Training data contain  $\mathcal{I}$ ,  $\rho$ , and target object parameters  $\mathbf{Y}_{\text{target}}$ . For selected objects, active coordinates are those requiring prediction, excluding padding and fixed coordinates. We initialize the active elements of  $\mathbf{Y}_0$  with independent noise from  $\mathcal{N}(0, 1)$  and sample  $t \sim \mathcal{U}[0, 1]$ . On the active coordinates, the interpolated state and flow loss are

$$\begin{aligned} \mathbf{Y}_t &= (1 - t)\mathbf{Y}_0 + t\mathbf{Y}_{\text{target}}, \\ \mathcal{L}_{\text{flow}} &= \mathbb{E} \left[ \|\mathbf{v}_t - (\mathbf{Y}_{\text{target}} - \mathbf{Y}_0)\|_{\text{act}}^2 \right]. \end{aligned} \quad (3)$$

Here,  $\|\cdot\|_{\text{act}}^2$  denotes mean squared error over active coordinates, with zero loss when no active coordinates exist. The expectation averages over the training data, noise, and flow time  $t$ . The parameters  $\theta$  of  $p_\theta(\pi | S, B)$  are optimized with the combined objective

$$\begin{aligned} \mathcal{L}(\theta) &= \mathcal{L}_{\text{order}} + \lambda_{\text{flow}} \mathcal{L}_{\text{flow}} \\ &\quad + \lambda_{\text{aff}} \mathcal{L}_{\text{aff}} + \lambda_{\text{avail}} \mathcal{L}_{\text{avail}}. \end{aligned} \quad (4)$$

The auxiliary loss  $\mathcal{L}_{\text{avail}}$  penalizes unavailable objects according to normalized first-interaction weights proportional to  $q_i \exp(-\mu_i)$ . Training labels determine availability, and  $\lambda_{\text{avail}} = 0$  when the auxiliary penalty is unused.

In the inference phase,  $\mathcal{I}$ ,  $\rho$ , and  $\mathbf{H}$  remain fixed throughout flow integration. Using the same rank-causal attention, we integrate  $d\mathbf{Y}_t/dt = \mathbf{v}_t$  from Gaussian noise  $\mathbf{Y}_0$  at  $t = 0$  to generate  $\mathbf{Y}_1$  at  $t = 1$ . The selected rows of  $\mathbf{Y}_1$  are decoded in the order  $\rho$  into  $(\xi_1, \dots, \xi_K)$  to construct the interaction plan  $\pi$ . For the same  $\mathcal{I}$  and  $\rho$ , different initial noise values allow the planner to generate alternative target configurations or object trajectories. The planner checks candidate plans for feasibility before execution (Sec. IV-D).

### D. Execution and Replanning

As shown in Fig. 2(d), candidate plans are checked for robot reachability, collision avoidance, and interaction constraints before execution. Validation follows  $\rho$  and considers the effects of preceding object motions, but is omitted in the Grid and Ordered Manipulation evaluations. Among feasible candidates satisfying the task goal  $G$ , the planner selects a plan with the minimum number of object interactions. A valid plan requiring no object interaction bypasses continuous generation. For legged embodiments, an MPC-based controller follows a policy-guided control structure [27], with command optimization adapted from [28]. Candidate command sequences are evaluated through rollouts of a learned locomotion policy to perform the planned object interactions. The optimization retains high-performing and behaviorally diverse command sequences, following a quality-diversity principle [10]. We combine this optimization with Cartesian tracking and joint-command smoothing (Fig. 3). The controller executes an initial segment of the best evaluated command sequence and updates the command optimization using new state estimates. After each object interaction, updated scene observations guide replanning of the remaining interactions.

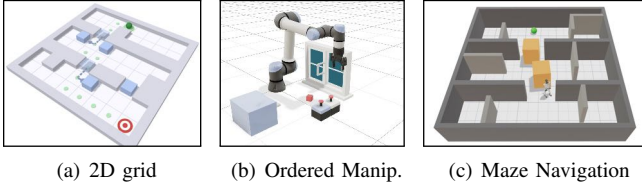


Fig. 4: **Experimental environments:** (a) 2D Grid, (b) Ordered Manipulation, and (c) Maze Navigation.

## V. EXPERIMENTS

In this section, we evaluate our proposed method across complementary task settings. The experiments address four questions. (1) Can the planner identify the required objects and recover executable minimum-interaction plans, including alternative optimal sequences? (2) Can the planner complete manipulation tasks while satisfying interaction preconditions? (3) How do embodiment information and learned affordances affect planning for different robots in the same scene? (4) When does closed-loop replanning improve task completion after environmental changes?

We conduct four experiments in three environments shown in Fig. 4: 2D Grid, Ordered Manipulation adapted from OGBench [29], and Maze Navigation. Training data comprise exact-search Grid solutions, policy demonstrations for Ordered Manipulation, and reference sequences and object trajectories for Maze Navigation. For Maze Navigation, embodiment-specific control policies are trained and fine-tuned using mujoco [30] and kept fixed across planning methods. At inference, the trained planning modules run on an NPU. Results are averaged across seeds, with  $\pm$  denoting the sample standard deviation (i.e., SD) when provided.

### A. 2D Grid Task

*a) Task and Setup:* The 2D Grid Task evaluates interaction-order dependence and alternative optimal sequences for reaching the goal with the fewest object interactions. Each interaction moves one object to a target cell, and navigation steps do not contribute to the interaction count. Scenes contain an agent, static walls, and movable objects on  $16 \times 16$ ,  $24 \times 24$ , or  $32 \times 32$  grids. We consider five scene families (Fig. 5). **Direct** requires no interaction, **Single-Gate** requires one, and **Chain** requires multiple interactions in a specific order. **Branch** allows alternative optimal sequences, while **Distractor** includes irrelevant objects.

*b) Baselines and Evaluation:* We compare CLEAR with three adapted baselines and two ordering variants, all trained on interaction plans obtained by exact search with additional training on separate scenes. **Autoregressive** [31] selects objects sequentially without repetition and terminates with a learned STOP token. **Gumbel-Sinkhorn** [32] jointly assigns objects to sequence positions and selects objects preceding STOP. **PolyNet** [33] generates alternative sequences by conditioning a shared autoregressive decoder on four distinct inputs, with supervised training replacing reinforcement learning. For each reference sequence, PolyNet uses the

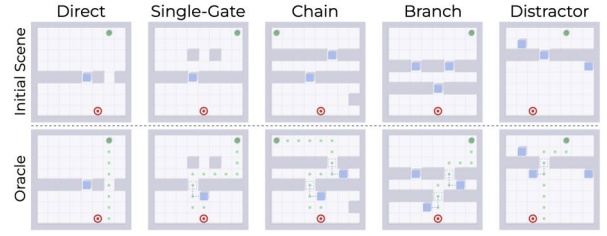


Fig. 5: **2D Grid scenes:** initial states (top) and an optimal plan per scene (bottom). Red marks goals; green, blue, and gray denote the agent, movable objects, and walls.

lowest prediction loss across the four inputs. **CLEAR-RO** randomly shuffles the predicted interaction order, whereas **CLEAR-HO** orders the same selected objects by increasing initial Manhattan distance.

CLEAR receives indicators of reachable pushing positions and allowed push directions for each object. For single-plan evaluation, our planner orders objects by mean predicted priority and selects the most frequently predicted target cell across four samples. Execution stops at the first infeasible interaction. **Task success rate (Task S/R)** is the percentage of scenes in which the agent reaches the goal. **Minimum-interaction success rate (Min. S/R)** additionally requires exactly  $K^*$  interactions, the minimum obtained by exact search. **Coverage (Cov.@8)** is the percentage of optimal object sequences recovered among the first eight proposals per scene, regardless of execution success. **Invalid rate (Inv.@8)** measures the execution failure rate among the first eight proposals per scene.

*c) Results and Analysis:* Our planner achieves the highest mean Task S/R (96.16%) and Min. S/R (96.02%), indicating that nearly all successful plans use the minimum number of interactions (Table I). Using random (**CLEAR-RO**) or distance-based (**CLEAR-HO**) interaction orders reduces mean Task S/R to 34.12% and 61.76%, respectively. The ordering ablations highlight the importance of preserving the planned interaction order. Our planner also achieves the highest mean Cov.@8 (88.11%) and lowest mean Inv.@8 (6.43%), supporting broader coverage of optimal object sequences alongside fewer execution failures.

### B. Ordered Manipulation Task

*a) Task and Setup:* We evaluate whether our planner completes manipulation tasks while satisfying interaction preconditions. In the adapted OGBench Scene environment [29], a UR5e arm interacts with a cube, two buttons, a drawer, and a sliding window. The scene observation specifies which buttons must be activated before moving the drawer or window. Placing the cube inside a closed drawer requires opening it, inserting the cube at a target determined by the drawer's intermediate position, and closing it. We train on seven combinations of button requirements and evaluate on two unseen combinations.

*b) Baselines and Evaluation:* We compare our planner with three baselines and two separately trained ablations.

| Method               | Task S/R $\uparrow$                | Min. S/R $\uparrow$                | Cov.@8 $\uparrow$                  | Inv.@8 $\downarrow$               |
|----------------------|------------------------------------|------------------------------------|------------------------------------|-----------------------------------|
| Autoregressive [31]  | 92.31 $\pm$ 0.64                   | 92.31 $\pm$ 0.64                   | 82.20 $\pm$ 0.21                   | 7.49 $\pm$ 0.98                   |
| Gumbel-Sinkhorn [32] | 91.81 $\pm$ 0.96                   | 79.77 $\pm$ 5.66                   | 72.20 $\pm$ 4.40                   | 9.38 $\pm$ 1.03                   |
| PolyNet [33]         | 89.49 $\pm$ 2.74                   | 89.49 $\pm$ 2.74                   | 85.95 $\pm$ 0.43                   | 16.88 $\pm$ 5.18                  |
| CLEAR-RO             | 34.12 $\pm$ 2.19                   | 34.12 $\pm$ 2.19                   | 61.93 $\pm$ 0.92                   | 64.33 $\pm$ 0.63                  |
| CLEAR-HO             | 61.76 $\pm$ 1.71                   | 61.67 $\pm$ 1.61                   | 59.51 $\pm$ 0.18                   | 39.21 $\pm$ 0.43                  |
| <b>CLEAR (Ours)</b>  | <b>96.16 <math>\pm</math> 0.67</b> | <b>96.02 <math>\pm</math> 0.55</b> | <b>88.11 <math>\pm</math> 0.32</b> | <b>6.43 <math>\pm</math> 0.59</b> |

TABLE I: **2D Grid results (%)**. Mean  $\pm$  sample SD over five training seeds on 432 test scenes. Cov.@8 and Inv.@8 use the first eight proposals per scene. Best means are bold.

**Classical Search** uses A\* search in Pyperplan [34] with manually specified preconditions. **Behavioral Cloning (BC)** [35] predicts the next object and corresponding target, while **BC-Grounding** uses our scene-grounding architecture with next-object prediction and independent target regression. **CLEAR w/o affordance** removes affordance conditioning, affordance supervision, and the availability loss, while **CLEAR w/o rank mask** removes the target generator’s rank-based attention mask. All learned methods use the same demonstration dataset for training and target-only refinement via least-squares refitting of the BC variants’ final regression layer or 6,000 additional flow updates for our planner and ablations.

All methods share an MPC-based controller and replan after each interaction, with up to 12 attempts including retries. After three consecutive failures at the same target with little change in object states, an alternative target is tried if available. **Task success rate (Task S/R)** measures the percentage of episodes with correct button states and cube, drawer, and window positions within 0.04m of the respective goals. **Valid success rate (Valid S/R)** additionally requires every attempted interaction to satisfy interaction preconditions. **Invalid episode rate (Invalid E/R)** measures the percentage of episodes with at least one interaction attempted with unmet preconditions.

*c) Results and Analysis:* **CLEAR** achieves the highest mean Task S/R (92.17%) and Valid S/R (91.92%), with a gap of 0.25 percentage points between the two metrics (Table II). **Classical Search** and **BC-Grounding** achieve lower Invalid E/R (0.83% and 0.67%, respectively), although both methods achieve lower Valid S/R than CLEAR. Removing affordance-related components reduces Task S/R by 2.50 and Valid S/R by 6.59 percentage points, while increasing Invalid E/R from 1.28% to 7.75%. The affordance ablation supports the role of conditioning and supervision in selecting interactions with satisfied preconditions. Removing the rank mask results in a smaller decrease in Valid S/R, from 91.92% to 90.33%, and increases Invalid E/R from 1.28% to 3.08%.

### C. Embodiment-Aware Maze Navigation

*a) Task and Setup:* We evaluate embodiment-conditioned navigation and object interaction in shared environments (Fig. 4(c)). We evaluate G1, Spot, Spot with an arm (Spot+Arm), and Husky, using differential-drive kinematics for Husky. The **Standard** set contains 15 scenes with varying terrain traversability and manipulation requirements, while **High-Mass** contains ten manipulation

| Method                | Task S/R $\uparrow$                | Valid S/R $\uparrow$               | Invalid E/R $\downarrow$          |
|-----------------------|------------------------------------|------------------------------------|-----------------------------------|
| Classical Search [34] | 83.80 $\pm$ 2.17                   | 83.80 $\pm$ 2.17                   | 0.83 $\pm$ 0.72                   |
| BC [35]               | 90.25 $\pm$ 1.78                   | 87.50 $\pm$ 2.59                   | 3.33 $\pm$ 2.72                   |
| BC-Grounding          | 87.42 $\pm$ 1.89                   | 87.42 $\pm$ 1.89                   | <b>0.67 <math>\pm</math> 1.09</b> |
| CLEAR w/o affordance  | 89.67 $\pm$ 3.50                   | 85.33 $\pm$ 5.71                   | 7.75 $\pm$ 5.29                   |
| CLEAR w/o rank mask   | 91.42 $\pm$ 1.97                   | 90.33 $\pm$ 2.29                   | 3.08 $\pm$ 3.20                   |
| <b>CLEAR (Ours)</b>   | <b>92.17 <math>\pm</math> 1.04</b> | <b>91.92 <math>\pm</math> 1.40</b> | 1.28 $\pm$ 2.44                   |

TABLE II: **Ordered Manipulation results (%)**. Mean  $\pm$  SD over five training seeds (learned methods) and three execution seeds (Classical Search). Best means are bold.

cases with 11.5 kg objects.

*b) Baselines and Evaluation:* We compare our planner with adapted **NAMO-LLM**<sup>1</sup> [36] and **CDGS** [37], which learns object-interaction transitions using our perception and terrain assessment components. **NAMO-LLM (w/o LLM)** uniformly samples objects passing geometric checks. **CLEAR-Specialists** trains one planner per embodiment under the same combined training budget as our shared model. **CLEAR-Robot-ID** and **CLEAR + GET** replace our structural encoder with a learned robot-identity embedding and an adapted Graph Embodiment Transformer [20], respectively. **CLEAR-No-Embodiment** removes embodiment and capability-conditioned navigation features, while **CLEAR-No-Affordance** replaces learned terrain guidance with geometric checks, retaining affordance supervision.

All methods use the same controller per embodiment and allow up to eight candidate plans and two object interactions without high-level replanning. **Success rate (S/R)** measures the percentage of episodes completing the task with final position changes below 0.20m for objects not targeted by interaction attempts. **Planning accuracy (P/A)** measures the percentage of cases where the planner produces a valid goal-reaching plan or correctly identifies the task as infeasible for the robot. We evaluate S/R for G1, Spot, and Spot+Arm, and P/A for Husky, with 45 Standard and 30 High-Mass trials per method and embodiment across three seeds.

*c) Results and Analysis:* Our planner achieves higher mean scores than the adapted external planners across all evaluated settings (Table III). The shared model achieves the highest Spot+Arm S/R in **Standard** (95.6%) and **High-Mass** (96.7%), and exceeds **CLEAR-Specialists** on High-Mass G1 (93.3% versus 73.3%), although Specialists perform best in several other comparisons. Our planner also exceeds **CLEAR-Robot-ID** and **CLEAR + GET** for Spot, Spot+Arm, and Husky in both sets, but not for G1. Removing embodiment information reduces High-Mass G1 S/R from 93.3% to 6.7% and Husky P/A from 100.0% to 10.0%. On Standard, **CLEAR-No-Affordance** achieves 60.0% Spot+Arm S/R compared with 95.6% for our planner, supporting the benefit of learned terrain guidance over geometric checks. Figure 6 illustrates how our planner generates different trajectories for G1, Spot, and Husky in the same

<sup>1</sup>NAMO-LLM uses GPT-4o for object selection within geometric relocation search. <https://platform.openai.com/docs/models/gpt-4o>

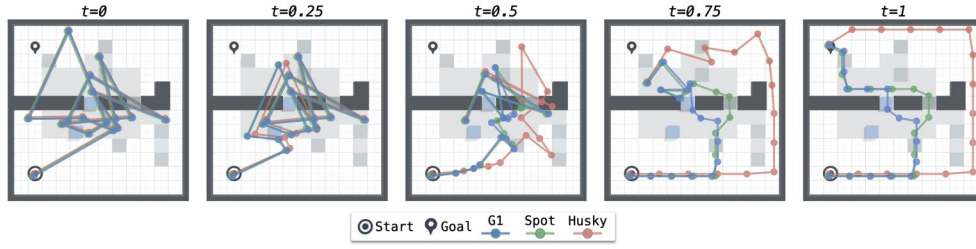


Fig. 6: **Embodiment-aware trajectory generation through conditional flow matching.** The panels show trajectory samples at successive flow times for G1 (blue), Spot (green), and Husky (red) in the same environment. Gaussian samples at  $t = 0$  evolve into distinct trajectories for each robot embodiment at  $t = 1$ . Here,  $t$  denotes flow time, which is distinct from physical execution time.

| Method              | Standard    |             |               |             | High-Mass   |             |               |              |
|---------------------|-------------|-------------|---------------|-------------|-------------|-------------|---------------|--------------|
|                     | G1 S/R↑     | Spot S/R↑   | Spot+Arm S/R↑ | Husky P/A↑  | G1 S/R↑     | Spot S/R↑   | Spot+Arm S/R↑ | Husky P/A↑   |
| NAMO-LLM [36]       | 64.4        | 53.3        | 13.3          | 55.6        | 26.7        | 26.7        | 0.0           | 20.0         |
| w/o LLM             | 62.2        | 44.4        | 13.3          | 46.7        | 40.0        | 20.0        | 0.0           | 13.3         |
| CDGS [37]           | 42.2        | 60.0        | 4.4           | 73.3        | 0.0         | 36.7        | 6.7           | 83.3         |
| CLEAR-Specialists   | <b>97.8</b> | <b>93.3</b> | 93.3          | <b>95.6</b> | 73.3        | <b>70.0</b> | 93.3          | <b>100.0</b> |
| CLEAR-Robot-ID      | 93.3        | 82.2        | 88.9          | 82.2        | <b>93.3</b> | 56.7        | 93.3          | 86.7         |
| CLEAR + GET [20]    | <b>97.8</b> | 86.7        | 75.6          | 86.7        | <b>93.3</b> | 60.0        | 86.7          | 90.0         |
| CLEAR-No-Affordance | 91.1        | 75.6        | 60.0          | 77.8        | 76.7        | 50.0        | 93.3          | 80.0         |
| CLEAR-No-Embodiment | 55.6        | 53.3        | 82.2          | 33.3        | 6.7         | 30.0        | 83.3          | 10.0         |
| <b>CLEAR (Ours)</b> | 91.1        | 91.1        | <b>95.6</b>   | 93.3        | <b>93.3</b> | 66.7        | <b>96.7</b>   | <b>100.0</b> |

TABLE III: **Embodiment-aware navigation.** Means (%) over three training seeds for learned methods and three execution seeds for NAMO-LLM variants. P/A denotes planning accuracy. Best means in each column are bold.

environment. The ablations support the contributions of embodiment conditioning and learned terrain guidance across the evaluated embodiments.

#### D. Long-Horizon Replanning

a) *Task and Setup:* This experiment evaluates recovery from environmental changes during long-horizon execution. We evaluate G1 in 12 MuJoCo scenes with six-interaction reference plans under three conditions: **Nominal**, **Placement**, and **Access**. In Nominal, execution proceeds without disturbance. In Placement, a manipulated object is moved away from the target position. In Access, another object is relocated to alter access to subsequent interactions.

b) *Baseline and Evaluation:* We compare **CLEAR-No-Replanning** and **CLEAR** using the same checkpoint, controller, low-level feedback, and execution limits. Both allow up to 14 interaction attempts, retrying failures once along the same object’s remaining planned path. **CLEAR-No-Replanning** follows the initial plan and stops after a failed retry, while **CLEAR** replans after successful interactions or failed retries. If replanning finds no valid candidate, the remaining plan is executed when available (i.e., residual fallback). Planning allows 15 calls, 300s total wall-clock time, and 60s per call. Success requires task completion without prohibited contact or wall penetration, with unintended object position changes below 0.10 m, excluding the external disturbance.

| Method              | Nominal           | Placement          | Access            |
|---------------------|-------------------|--------------------|-------------------|
| CLEAR-No-Replanning | 83.3 ± 4.8        | 81.9 ± 0.0         | 50.0 ± 14.4       |
| <b>CLEAR (Ours)</b> | <b>88.9 ± 4.8</b> | <b>87.2 ± 14.4</b> | <b>91.7 ± 8.3</b> |

TABLE IV: **Replanning under environmental changes.** Success rates (%; mean ± SD) for G1 on 12 MuJoCo scenes with three execution seeds. Best means are bold.

c) *Results and Analysis:* **CLEAR** achieves higher mean S/R in all three conditions (Table IV). The improvement is largest in the **Access** scenario with S/R increasing from 50.0% to 91.7%. S/R increases by 5.6 and 5.3 percentage points in the **Nominal** and **Placement** scenarios, respectively. These results support updating the remaining plan when environmental changes alter object placement or the reachability of subsequent interaction locations.

## VI. CONCLUSION

We present **CLEAR**, a capability-aware approach to planning coupled object selection, interaction order, and object motion. Affordance-guided scene grounding connects these decisions to robot capabilities, while latent ordering represents alternative sequences and rank-causal generation conditions object motions on preceding interactions. The experiments demonstrate executable minimum-interaction planning in grids, closed-loop manipulation under changing preconditions, and capability-conditioned navigation with a shared planning model. The long-horizon evaluation demonstrates recovery when environmental changes invalidate the remaining plan. Future work will extend sequence-dependent object-motion generation beyond the current interaction representations and validate the plans on physical robots.

## REFERENCES

- [1] M. Stilman and J. J. Kuffner, “Navigation among movable obstacles: Real-time reasoning in complex environments,” *International Journal of Humanoid Robotics*, vol. 2, no. 4, pp. 479–503, 2005. [Online]. Available: <https://doi.org/10.1142/S0219843605000545>
- [2] Z. Ren, B. Suvonov, G. Chen, B. He, Y. Liao, C. Fermuller, and J. Zhang, “Search-based path planning in interactive environments among movable obstacles,” in *IEEE International Conference on Robotics and Automation*. IEEE, 2025, pp. 533–539. [Online]. Available: <https://doi.org/10.1109/icra55743.2025.11128242>
- [3] M. Stilman and J. Kuffner, “Planning among movable obstacles with artificial constraints,” *The International Journal of Robotics Research*, vol. 27, no. 11–12, pp. 1295–1307, 2008. [Online]. Available: <https://doi.org/10.1177/0278364908098457>

- [4] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 9902–9915. [Online]. Available: <https://proceedings.mlr.press/v162/janner22a.html>
- [5] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. C. Burchfiel, and S. Song, "Diffusion Policy: Visuomotor policy learning via action diffusion," in *Robotics: Science and Systems*, 2023. [Online]. Available: <https://www.roboticsproceedings.org/rss19/p026.html>
- [6] B. He, G. Chen, W. Wang, J. Zhang, C. Fermuller, and Y. Aloimonos, "Interactive-FAR: Interactive, fast and adaptable routing for navigation among movable obstacles in complex unknown environments," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2024, pp. 5402–5409. [Online]. Available: <https://doi.org/10.1109/iro58592.2024.10802169>
- [7] A. Krontiris and K. Bekris, "Dealing with difficult instances of object rearrangement," in *Robotics: Science and Systems*, 2015. [Online]. Available: <https://www.roboticsproceedings.org/rss11/p45.html>
- [8] J. Ahn and C. Mavrogiannis, "ReloPush: Multi-object rearrangement in confined spaces with a nonholonomic mobile robot pusher," in *IEEE International Conference on Robotics and Automation*. IEEE, 2025, pp. 13 196–13 203. [Online]. Available: <https://doi.org/10.1109/icra55743.2025.11128108>
- [9] J. Lee, S. Park, J. Park, K. Lee, and S. Choi, "SPOTS: Stable placement of objects with reasoning in semi-autonomous teleoperation systems," in *IEEE International Conference on Robotics and Automation*, 2024, pp. 17 786–17 792.
- [10] S. Park, T. Yoon, J. Lee, S. Park, and S. Choi, "Quality-diversity based semi-autonomous teleoperation using reinforcement learning," *Neural Networks*, 2024. [Online]. Available: <https://doi.org/10.1016/j.neunet.2024.106543>
- [11] W. Liu, Y. Du, T. Hermans, S. Chernova, and C. Paxton, "StructDiffusion: Language-guided creation of physically-valid structures using unseen objects," in *Robotics: Science and Systems*, 2023. [Online]. Available: <https://www.roboticsproceedings.org/rss19/p031.html>
- [12] Z. Yang, J. Mao, Y. Du, J. Wu, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, "Compositional diffusion-based continuous constraint solvers," in *Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 3242–3265. [Online]. Available: <https://proceedings.mlr.press/v229/yang23d.html>
- [13] U. A. Mishra, S. Xue, Y. Chen, and D. Xu, "Generative skill chaining: Long-horizon skill planning with diffusion models," in *Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 2905–2925. [Online]. Available: <https://proceedings.mlr.press/v229/mishra23a.html>
- [14] U. A. Mishra, Y. Chen, and D. Xu, "Generative factor chaining: Coordinated manipulation with diffusion-based factor graph," in *Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 1461–1472. [Online]. Available: <https://proceedings.mlr.press/v270/mishra25a.html>
- [15] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *International Conference on Learning Representations*, 2023. [Online]. Available: <https://openreview.net/forum?id=PqvMRDCJT9t>
- [16] T. Wang, R. Liao, J. Ba, and S. Fidler, "NerveNet: Learning structured policy with graph neural networks," in *International Conference on Learning Representations*, 2018. [Online]. Available: <https://openreview.net/forum?id=S1sqHMZCb>
- [17] W. Huang, I. Mordatch, and D. Pathak, "One policy to control them all: Shared modular policies for agent-agnostic control," in *International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 4455–4464. [Online]. Available: <https://proceedings.mlr.press/v119/huang20d.html>
- [18] A. Gupta, L. Fan, S. Ganguli, and L. Fei-Fei, "MetaMorph: Learning universal controllers with transformers," in *International Conference on Learning Representations*, 2022. [Online]. Available: [https://openreview.net/forum?id=OpmqtK\\_GvYL](https://openreview.net/forum?id=OpmqtK_GvYL)
- [19] C. Sferazza, D.-M. Huang, F. Liu, J. Lee, and P. Abbeel, "Body Transformer: Leveraging robot embodiment for policy learning," in *Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 3407–3424. [Online]. Available: <https://proceedings.mlr.press/v270/sferazza25a.html>
- [20] A. Patel and S. Song, "GET-Zero: Graph embodiment transformer for zero-shot embodiment generalization," in *IEEE International Conference on Robotics and Automation*. IEEE, 2025, pp. 14 262–14 269. [Online]. Available: <https://doi.org/10.1109/icra55743.2025.11127922>
- [21] M. Xu, Z. Xu, C. Chi, M. Veloso, and S. Song, "XSkill: Cross embodiment skill discovery," in *Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 3536–3555. [Online]. Available: <https://proceedings.mlr.press/v229/xu23a.html>
- [22] J. H. Yang, C. Glossop, A. Bhorkar, D. Shah, Q. Vuong, C. Finn, D. Sadigh, and S. Levine, "Pushing the limits of cross-embodiment learning for manipulation and navigation," in *Robotics: Science and Systems*, 2024. [Online]. Available: <https://www.roboticsproceedings.org/rss20/p093.html>
- [23] J. Park, S. Lim, J. Lee, S. Park, M. Chang, Y. Yu, and S. Choi, "CLARA: Classifying and disambiguating user commands for reliable interactive robotic agents," *IEEE Robotics and Automation Letters*, vol. 9, no. 2, pp. 1059–1066, 2024.
- [24] Z. Wu, R. Tong, J. Fang, and T. Bhattacharjee, "Embodiment meets environment: Toward context-aware, safe physical caregiving robots," in *Robotics: Science and Systems*, 2026. [Online]. Available: <https://www.roboticsproceedings.org/rss22/p117.html>
- [25] J. C. Bean, "Genetic algorithms and random keys for sequencing and optimization," *ORSA Journal on Computing*, vol. 6, no. 2, pp. 154–160, 1994. [Online]. Available: <https://pubsonline.informs.org/doi/abs/10.1287/ijoc.6.2.154>
- [26] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I. the method of paired comparisons," *Biometrika*, vol. 39, no. 3–4, pp. 324–345, 1952. [Online]. Available: <https://doi.org/10.1093/biomet/39.3-4.324>
- [27] J. Z. Zhang, M. Sorokin, J. Brüdigam, B. Hung, S. Phillips, D. Yershov, F. Niroui, T. Zhao, L. Fermoselle, X. Zhu, C. Cao, D. Ta, T. Pang, J. Wang, P. Culbertson, Z. Manchester, and S. L. Cléac'h, "Sumo: Dynamic and generalizable whole-body loco-manipulation," *arXiv preprint arXiv:2604.08508*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.08508>
- [28] A. H. Li, B. Hung, A. D. Ames, J. Wang, S. L. Cléac'h, and P. Culbertson, "Judo: A user-friendly open-source package for sampling-based model predictive control," in *Workshop on Fast Motion Planning and Control in the Era of Parallelism at Robotics: Science and Systems*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.17184>
- [29] S. Park, K. Frans, B. Eysenbach, and S. Levine, "OGBench: Benchmarking offline goal-conditioned RL," in *International Conference on Learning Representations*, 2025. [Online]. Available: <https://iclr.cc/virtual/2025/poster/29950>
- [30] K. Zakka, Q. Liao, B. Yi, L. L. Lay, K. Sreenath, and P. Abbeel, "mjlab: A lightweight framework for GPU-accelerated robot learning," *arXiv preprint arXiv:2601.22074*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.22074>
- [31] W. Kool, H. van Hoof, and M. Welling, "Attention, learn to solve routing problems!" in *International Conference on Learning Representations*, 2019.
- [32] G. Mena, D. Belanger, S. Linderman, and J. Snoek, "Learning latent permutations with Gumbel-Sinkhorn networks," in *International Conference on Learning Representations*, 2018.
- [33] A. Hottung, M. Mahajan, and K. Tierney, "PolyNet: Learning diverse solution strategies for neural combinatorial optimization," in *International Conference on Learning Representations*, 2025.
- [34] Y. Alkhazraji, M. Frorath, M. Grütznier, M. Helmert, T. Liebetraut, R. Mattmüller, M. Ortlieb, J. Seipp, T. Springenberg, P. Stahl, and J. Wülfing, "Pyperplan," <https://doi.org/10.5281/zenodo.3700819>, 2020.
- [35] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters, "An algorithmic perspective on imitation learning," *Foundations and Trends in Robotics*, vol. 7, no. 1–2, pp. 1–179, 2018.
- [36] Y. Zhang and Y. Kantaros, "NAMO-LLM: Efficient navigation among movable obstacles with large language model guidance," *IEEE Robotics and Automation Letters*, vol. 10, no. 12, pp. 13 026–13 033, 2025.
- [37] U. A. Mishra, D. He, Y. Chen, and D. Xu, "Compositional diffusion with guided search for long-horizon planning," in *International Conference on Learning Representations*, 2026. [Online]. Available: <https://openreview.net/forum?id=b8avf4F2hn>